SILOSTITCH: Integrating Pretrained Intrusion Detectors under Deployment Budgets

***Abstract*—Organizations may retain intrusion detectors after storage or retention policies remove the records that trained them. When a deployment cannot afford the entire retained pool, a ranking based only on current average utility can omit the sole feasible detector that supports an important attack class. Yet historical support alone cannot show whether a detector remains competent. We present SILOSTITCH, which integrates unchanged pretrained detectors under a hard logical byte budget. Before selection, the method aligns each detector's probability outputs and uses records reserved for calibration to assess current performance. The coordinator then uses dynamic programming to maximize weighted source class support exactly and compares current behavior only among sets that attain this value. After selection, the coordinator fits one shared stacker on a disjoint partition. Clients may add Laplace noise to selected score blocks before upload. We prove that simultaneous calibration remains valid after adaptive selection and establish a local certificate for the secondary refinement. Under a neighboring relation that fixes the public label, the Laplace release gives each uploaded score block label conditional record level local differential privacy. Across four cybersecurity datasets, a 50% target budget reduces logical payload by between 49.3% and 50.4%. At this budget, macro F1 stays within 0.0052 of the full pool on MalDroid, Darknet, and DoH, while DDoS shows a 0.0258 gap.**

I. INTRODUCTION

Because storage or retention policies may remove historical traffic logs [1], [2], operational networks often keep intrusion detectors longer than the records that trained them. As attackers and benign applications change their behavior, deployed security models face a shifting data distribution [3]. Detectors from different organizations therefore retain different attack knowledge and current prediction behavior. Organizations could retrain the detectors collaboratively only if source data remained available and contributors permitted model updates [4]–[6]. We therefore study how organizations can integrate pretrained detectors while leaving their parameters unchanged.

When a deployment cannot run every contributed detector, the coordinator must choose a subset under a hard budget. It must pay to distribute each model and process its probability block. If the coordinator ranks detectors only by average current utility, it may omit the sole feasible model that supports a less frequent or important attack class. By contrast, if the coordinator relies only on historical source support, it may preserve nominal breadth with detectors that perform poorly or repeat behavior already chosen.

To resolve this conflict, SILOSTITCH treats source support and current behavior as ordered decisions. The coordinator first preserves the largest weighted source class support that the budget permits. Only after it fixes that value does it compare current utility and representative behavior. Because an instance can contain an arbitrarily small positive support gap, no fixed weighted sum can enforce this priority for every candidate pool.

Before selection, each client reorganizes detector outputs into the same target class order and reserves one entry for unsupported labels. The method then uses records reserved for calibration to estimate overall utility, conservative class utility, and prediction behavior. At the same time, fixed source counts record which classes trained each detector, while a logical byte cost accounts for model delivery and the score block used by the shared stacker. These inputs keep historical support, current competence, and deployment cost separate.

After calibration, the coordinator rejects candidates that fail the eligibility gate. It uses dynamic programming to compute the best attainable source support exactly. It then refines current behavior without lowering that value and uses remaining capacity through feasible additions or single model replacements. Once selection ends, it trains one shared stacker on a disjoint fit partition of the selected probability blocks [7].

Because the workflow reserves separate records for candidate fitting, selection calibration, stacker fitting, and sealed testing, test labels do not influence selection or training. When clients enable score protection, they add fresh Laplace noise to each selected score block before upload. Under a neighboring relation that fixes the public label, this release gives each block label conditional record level local differential privacy and composes across the released blocks.

We prove that simultaneous calibration remains valid after adaptive selection. The primary program attains the maximum weighted support. The refinement preserves that value and returns a locally stable set. We also characterize when current class utility continues to support historical class claims as the client mixture changes.

We evaluate SILOSTITCH on four cybersecurity datasets with 66 candidate detectors per task. At a 50% target budget, the method reduces logical payload by between 49.3% and 50.4%. It keeps macro F1 within 0.0052 of the full pool on MalDroid, Darknet, and DoH, while DDoS shows a 0.0258 gap. Across 40 comparisons on small instances, the primary solver always matches exhaustive search. Across seven Laplace settings, the sensitivity experiment reveals how score randomization changes detection quality.

We make three contributions.

- We formulate budgeted detector integration after training as an ordered decision that separates source class support from current predictive evidence.

- We design SILOSTITCH to align frozen detectors, compute the exact maximum weighted support, preserve that support during local refinement, and fit one shared stacker from optionally perturbed scores.
- We establish calibration bounds that remain valid after adaptive selection, distinguish exact primary optimization from local secondary refinement, prove label conditional record level local differential privacy for uploaded score blocks, and analyze class support as the client mixture changes.

II. BACKGROUND AND RELATED WORK

This section distinguishes post-training detector integration from collaborative retraining, reviews nearby integration and selection methods, and introduces the privacy notion used for score uploads.

A. Detector Retraining and Adaptation

Federated learning coordinates parameter updates while keeping raw records at their owners, which suits settings in which participants retain training data and can run another optimization process [4]–[6]. Within intrusion detection, federated methods adapt this process to non-IID traffic and heterogeneous local learners [8], [9]. Other recent NIDS methods revise the training pipeline under difficult data conditions. RAPIER handles low quality labels for encrypted malicious traffic [10]. Rosetta uses TCP aware augmentation to improve robustness across network environments [11]. CoLD denoises labels before fitting a classifier [12]. Our setting instead begins after the contributed detectors have been trained and their historical records are no longer available.

B. Pretrained Model Integration

When source records disappear but their trained models remain accessible, pretrained-model integration provides the closest starting point. Existing approaches either rank a heterogeneous model zoo by cross-domain transferability and aggregate its top K models [13] or combine pretrained models without accessing their source records [14], [15]. They do not select one fixed detector subset under a hard aggregate budget while treating class support as a strict priority.

C. Output-Level Ensembles and Cost-Aware Selection

Once model outputs are available, stacked generalization can train a downstream classifier over them [7]. In security, SIRAJ aggregates reports from heterogeneous threat intelligence sources by pretraining an embedding and fine tuning downstream predictors [16], while Fed-MStacking trains a global meta-classifier from heterogeneous local predictions in a missing-label setting [17]. A separate line studies cost-aware inference: SPIREL learns a per-item policy that trades off model invocations and classification utility [18]. These approaches either combine outputs or control per-item cost, but they do not screen detectors and return one fixed subset under a hard aggregate budget. SILOSTITCH makes this subset decision before fitting one shared classifier over the selected outputs.

D. Local Privacy for Prediction Uploads

Prediction outputs can reveal information about their inputs, so a privacy guarantee must identify both where randomization occurs and which fields it covers. PrivateKT perturbs locally produced predictions before sending them for knowledge transfer [19]. Differentially private federated trees instead protect statistics exchanged while training a new model [20]. Because SILOSTITCH uploads selected detector scores to fit a shared classifier while treating labels as public, we specialize the local model of Duchi et al. [21] to this release.

Definition II.1 (Label-conditional local differential privacy)**.** For a fixed public label y, a client applies a randomized mechanism $\mathcal{M}_y$ to the feature vector x. The upload $\mathcal{R}(x, y) = (\mathcal{M}_y(x), y)$ satisfies label-conditional $(\epsilon, 0)$-local differential privacy if, for every fixed y, every pair x, x', and every measurable output set $\mathcal{O}$,

$$\Pr[\mathcal{M}_y(x) \in \mathcal{O}] \leq e^{\epsilon} \Pr[\mathcal{M}_y(x') \in \mathcal{O}]. \tag{1}$$

The definition compares records with the same public label and protects the randomized, feature-derived output.

To instantiate Definition II.1, let a deterministic score map g have replacement sensitivity

$$\Delta_1(g) = \sup_{x,x'} \|g(x) - g(x')\|_1. \tag{2}$$

Adding independent noise $\eta_k \sim \mathrm{Lap}(0, \Delta_1(g)/\epsilon)$ to every coordinate satisfies Definition II.1 [22]. If one record releases blocks with parameters $\epsilon_1, \ldots, \epsilon_s$, sequential composition gives parameter $\sum_{i=1}^{s} \epsilon_i$. Under one-record replacement, releases derived from unchanged records do not add to this privacy parameter.

In SILOSTITCH, this mechanism applies to the selected detector-score blocks. Theorem V.1 establishes label-conditional local privacy for that score upload under the public-label adjacency relation.

E. Capability Comparison

Table I summarizes the closest methods along eight concrete capabilities. The integration columns describe whether a method accepts frozen heterogeneous models and stacks their outputs. The selection columns describe whether it screens detectors on current data and returns one fixed set under a hard budget while preserving attainable class support as a strict priority. The final column records whether predictions are randomized locally before upload.

A partial mark denotes related but non-equivalent functionality. SIRAJ learns from scanner reports instead of accepting frozen model objects. ZooD ranks models with evidence from held out source domains and aggregates a fixed set of its top K models. Its rule uses a fixed cardinality instead of an aggregate cost limit. SPIREL incorporates cost into a per-item policy rather than returning one fixed detector set, while PrivateKT aggregates randomized predictions instead of fitting a fixed score stacker. Fed-MStacking addresses missing labels without treating class support as a strict selection priority. These

Table I
COMPARISON WITH RECENT RELATED METHODS. HERE, ✓ DENOTES DIRECT SUPPORT, ● RELATED SUPPORT, AND ○ NO SUPPORT.

Method	Model Integration			Subset Selection				Privacy
	Frozen Models	Heterogeneous	Score Stacking	Current Screening	Fixed Set	Hard Budget	Class Priority	Score LDP
SIRAJ [16]	○	✓	●	○	○	○	○	○
PrivateKT [19]	○	○	●	○	○	○	○	✓
SPIREL [18]	✓	✓	●	○	●	●	○	○
ZooD [13]	✓	✓	●	○	✓	●	○	○
Fed-MStacking [17]	○	✓	✓	○	○	○	○	○
SILOSTITCH	✓	✓	✓	✓	✓	✓	✓	✓

distinctions motivate the ordered selection problem formalized next.

III. PROBLEM STATEMENT

We formalize post-training detector integration, define the ordered selection objective, and derive its selection and privacy guarantees.

A. System Model

After local detector fitting has ended, J clients expose a pool of M candidate detectors. Our experiments assign one candidate to each client, while the method allows J and M to vary independently. For candidate i, let $p_i(x) \in \mathbb{R}^{h_i}$ denote its native probability output. Because different candidates may use different native label coordinates, the method validates a hard semantic map

$$A_i \in \{0,1\}^{(C+1)\times h_i}, \qquad \mathbf{1}^\top A_i = \mathbf{1}^\top, \tag{3}$$

and forms

$$q_i(x) = A_i p_i(x) \in \Delta^C, \tag{4}$$

where Δ^C is the probability simplex with $C+1$ coordinates. The first C coordinates follow a frozen target ontology, whereas the last coordinate, denoted by $\perp$, collects unsupported outputs. Thus, any mass assigned to $\perp$ remains visible to every later Brier calculation.

Before selection begins, the method freezes every candidate, semantic map, and source-side class count. It also assigns disjoint record partitions to candidate fitting, selection calibration, stacker fitting, and sealed testing. Because test labels remain unopened until both the selected set and stacker have been frozen, they cannot affect calibration, cost, or selection.

The selection stage uses a logical cost for each selected candidate. Let K_i be candidate i's serialized size, let B_t be the number of stacker-fit records, and let d be the logical byte width of one probability element. For a selected set S, define

$$C_{\mathrm{sel}}(S) = \sum_{i\in S} b_i, \qquad b_i = JK_i + dB_t(C+1). \tag{5}$$

The first term delivers a selected candidate to J clients, and the second uploads one aligned probability block per stacker-fit record. Given a budget fraction β, we set

$$B = \left\lfloor \beta \sum_{i=1}^{M} b_i \right\rfloor. \tag{6}$$

The denominator contains every candidate, including those that may later fail calibration. The total logical communication measured in the evaluation also includes the initial candidate upload and the stacker label upload:

$$C_{\mathrm{total}}(S) = \sum_{i=1}^{M} K_i + C_{\mathrm{sel}}(S) + dB_t. \tag{7}$$

We use C_{sel} to enforce the budget and C_{total} to compare the complete communication of selected and full pool configurations.

B. Detector Selection Objective

With the aligned detector pool and logical budget defined, we next specify what the selector should optimize.

If the coordinator ranks candidates only by average current utility, it may omit the sole contributor that supports an important class. Source-side support alone has the opposite weakness: it can retain candidates whose current probabilities are weak or redundant. A scalar tradeoff can exchange a positive coverage loss for enough secondary gain, so the method gives weighted source-side coverage strict priority and compares current behavior only after that primary value has been fixed.

Accordingly, the intended target is

$$\mathrm{lexmax}_{\varnothing \neq S \subseteq \widehat{\mathcal{R}}} \left(C_{\mathrm{sem}}(S), \Phi(S)\right) \quad \text{s.t.} \quad \sum_{i\in S} b_i \le B. \tag{8}$$

Here, $\widehat{\mathcal{R}}$ contains candidates that pass calibration, C_{sem} measures their weighted source support, and Φ summarizes current predictive evidence. The following sections define each quantity. Equation 8 specifies the decision order. The primary solver computes its first coordinate exactly and refines the secondary coordinate heuristically. Only the optional small-pool exhaustive solver computes the complete lexicographic optimum.

C. Execution and Analysis Model

We next specify the execution model and privacy setting used by the analysis.

We evaluate the workflow in a centralized simulation. Figure 1 encodes the protocol's logical data dependencies. In a client-side realization, each client perturbs its selected probability blocks with fresh private Laplace randomness

Algorithm 1 SILOSTITCH Client–Server Procedure

Require: $\{f_i, A_i, H_i, K_i\}_{i=1}^{M}$, $\mathcal{D}^{\mathrm{cal}}$, $\{\mathcal{D}_j^{\mathrm{stk}}\}_{j=1}^{J}$, B, $\epsilon_b \in (0, \infty]$
Ensure: $\widehat{S}$, $\widehat{f}$

Clients $\to$ Server: $\{f_i, A_i, H_i, K_i\}_{i=1}^{M}$
$(\widehat{\mathcal{R}}, \Gamma, w, b, \Phi) \leftarrow \textsc{Calibrate}(\mathcal{D}^{\mathrm{cal}})$
$(W^\star, S_0) \leftarrow \textsc{Cover}(\widehat{\mathcal{R}}, \Gamma, w, b, B)$
$\widehat{S} \leftarrow \textsc{Refine}(S_0, W^\star, \Phi, B)$
Server $\to$ Clients: $\{f_i, A_i\}_{i \in \widehat{S}}$
for $j = 1, \ldots, J$ **in parallel do**
    $Z_j \leftarrow \mathrm{concat}_{i \in \widehat{S}}\, q_i(X_j^{\mathrm{stk}})$
    $(\Xi_j)_{t,i,k} \overset{\mathrm{iid}}{\sim} \mathrm{Lap}(0, 2/\epsilon_b)$
    $\widetilde{Z}_j \leftarrow Z_j + \Xi_j$
    Client $j \to$ Server: $(\widetilde{Z}_j, Y_j^{\mathrm{stk}})$
end for
$\widehat{f} \leftarrow \textsc{Fit}(\bigcup_j (\widetilde{Z}_j, Y_j^{\mathrm{stk}}))$
return $(\widehat{S}, \widehat{f})$

before upload. The optional sensitivity mode samples the same distribution centrally to measure utility.

The privacy analysis uses public labels and protects the randomized score blocks used to fit the stacker. It assumes immutable candidate detectors, structurally validated semantic maps, honest source-support reports, and fresh client-side randomness. These assumptions define the cooperative-client execution model analyzed below.

IV. Design of SiloStitch

In this section, we describe how SILOSTITCH calibrates the frozen candidate pool, selects a budget-feasible detector set, and trains one shared stacker.

A. Selection and Stacking Workflow

Figure 1 separates the records used for detector selection from those used for stacker fitting. Calibration outputs determine eligibility and the selected set, whereas stacker-fit features and score blocks do not enter selection. Clients use them only after that set has been frozen and then upload only the selected aligned blocks, optionally perturbed, and their labels; no contributed detector is updated.

With these stages in place, Algorithm 1 summarizes the complete client–server procedure, while the following subsections define CALIBRATE, COVER, and REFINE in detail. Here, ϵ_b is the privacy parameter assigned to one selected detector block; $\epsilon_b = \infty$ disables perturbation, so $\mathrm{Lap}(0, 0)$ denotes zero noise.

B. Probability Alignment and Brier Calibration

Following the workflow overview, we first specify how aligned outputs are calibrated and summarized.

For calibration record (x_t, y_t), let $e_{y_t} \in \mathbb{R}^{C+1}$ be the one-hot label vector with a zero $\perp$ coordinate. We evaluate each candidate with the complete aligned output and define

$$\ell_{i,t} = \frac{1}{2} \left\| q_i(x_t) - e_{y_t} \right\|_2^2, \qquad g_{i,t} = 1 - \ell_{i,t} \in [0, 1]. \tag{9}$$

Thus, $g_{i,t}$ is one full-vector Brier utility rather than a collection of coordinate-wise scores.

Eligibility screening. To screen out candidates whose average current loss is either too large or too uncertain, the method uses N calibration records and failure level $\delta \in (0, 1)$ to compute

$$\widehat{\mu}_i = \frac{1}{N} \sum_{t=1}^{N} g_{i,t}, \qquad r = \sqrt{\frac{\log(4M/\delta)}{2N}}, \tag{10}$$

and admits candidate i only if

$$R_i^+ = 1 - \widehat{\mu}_i + r \le \tau_L. \tag{11}$$

Accordingly, the calibration-eligible pool is

$$\widehat{\mathcal{R}} = \{i : R_i^+ \le \tau_L\}. \tag{12}$$

Thus, eligibility depends on an upper confidence bound on risk rather than on empirical utility alone.

Classwise evidence. Average eligibility does not preserve class-specific evidence for later refinement and certification. For class c, let $N_c = \sum_t \mathbf{1}\{y_t = c\}$ and

$$\widehat{\mu}_{i,c} = \frac{1}{N_c} \sum_{t : y_t = c} g_{i,t}. \tag{13}$$

Because the protocol requires every calibration class to occur, $N_c > 0$. The class-conditional radius and lower utility are

$$r_c = \sqrt{\frac{\log(4MC/\delta)}{2N_c}}, \qquad a_{i,c} = \left[\widehat{\mu}_{i,c} - r_c\right]_+, \tag{14}$$

where $[z]_+ = \max\{0, z\}$. Hence, $a_{i,c}$ is the class-specific lower score used by the secondary objective and the effective-coverage certificate.

Behavioral similarity. Two eligible candidates can still behave almost identically. To measure this redundancy, the method deterministically selects a class-stratified anchor set $\mathcal{A}$ and forms

$$z_i = \mathrm{vec}\left([q_i(x_t)]_{t \in \mathcal{A}}\right), \qquad \kappa_{i,k} = \left[\frac{\langle z_i, z_k \rangle}{\|z_i\|_2 \|z_k\|_2}\right]_0^1. \tag{15}$$

Here, $[\cdot]_0^1$ clips numerical roundoff into $[0, 1]$. We use $\kappa_{i,k}$ in a facility-location term that measures representativeness, not as a pairwise-diversity score.

C. Coverage-Prioritized Detector Selection

Calibration now supplies the evidence needed for budgeted selection. We first define the primary support objective.

Weighted semantic coverage. For candidate i and target class c, let $H_{i,c}$ be the accepted source-side support count after semantic mapping. After fixing $n_{\min} > 0$, define

$$\Gamma_{i,c} = \mathbf{1}\{H_{i,c} \ge n_{\min}\}. \tag{16}$$

If $H_c = \sum_i H_{i,c} > 0$, we define

$$w_c = \frac{H_c^{-1/2}}{\sum_{d=1}^{C} H_d^{-1/2}}, \qquad \sum_{c=1}^{C} w_c = 1. \tag{17}$$

Thus, classes with less aggregate source support receive greater weight. Then, for any selected set S,

$$C_{\mathrm{sem}}(S) = \sum_{c=1}^{C} w_c \mathbf{1}\{\exists i \in S : \Gamma_{i,c} = 1\}. \tag{18}$$

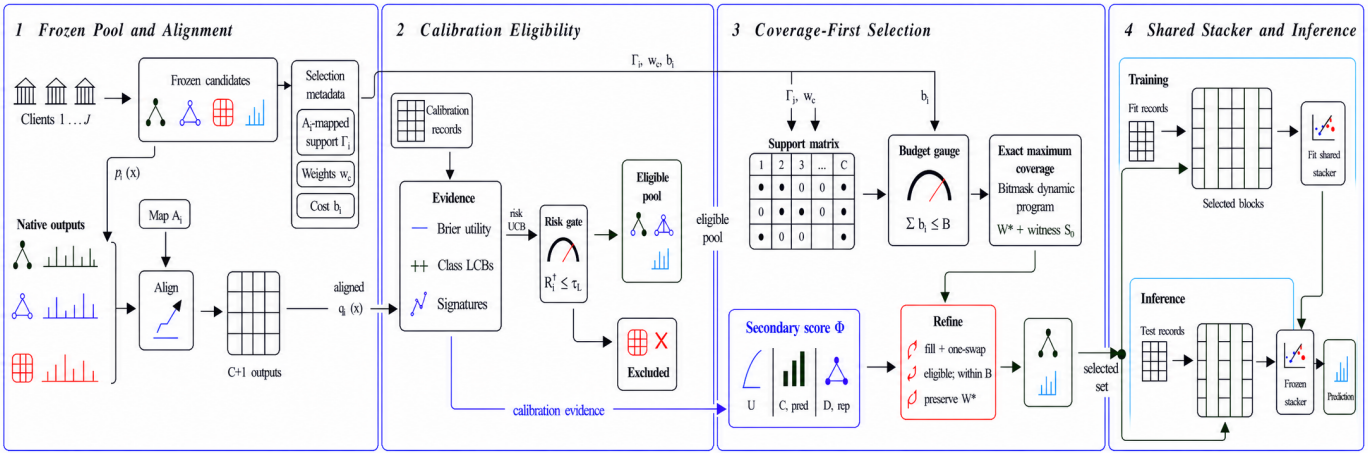

Figure 1. SILOSTITCH first freezes and aligns the candidate detector pool. It then uses selection-calibration records to form the eligible pool, obtains the exact maximum weighted semantic coverage under the logical budget, and performs coverage-preserving local secondary refinement. Finally, selected aligned probability blocks train one shared stacker and support inference. Arrows encode the protocol's logical data dependencies.

The $\perp$ coordinate never contributes to this coverage. Consequently, C_{sem} measures the breadth of declared source support rather than current predictive accuracy.

Secondary behavioral objective. Only after fixing semantic coverage does the method compare current predictive behavior. With $h_\tau(z) = 1 - e^{-z/\tau}$, define

$$U(S) = \frac{1}{N}\sum_{t=1}^{N} h_{\tau_U}\left(\sum_{i\in S} g_{i,t}\right),$$
$$C_{\text{pred}}(S) = \frac{1}{C}\sum_{c=1}^{C} h_{\tau_C}\left(\sum_{i\in S} a_{i,c}\right),$$
$$D_{\text{rep}}(S) = \frac{1}{|\widehat{\mathcal{R}}|}\sum_{k\in\widehat{\mathcal{R}}} \max_{i\in S} \kappa_{i,k}. \tag{19}$$

Here, U summarizes record-level utility, C_{pred} summarizes conservative classwise evidence, and D_{rep} rewards behavior that represents the eligible pool. Here and throughout the analysis, we set $\max_{i\in\varnothing}\kappa_{i,k} = 0$. Unlike C_{sem}, C_{pred} averages classes uniformly. After fixing nonnegative weights that sum to one, the secondary objective is

$$\Phi(S) = \lambda_U U(S) + \lambda_C C_{\text{pred}}(S) + \lambda_D D_{\text{rep}}(S). \tag{20}$$

This score is used only after semantic coverage has been fixed.

Budgeted selection procedure. We therefore define the feasible family and its optimal primary value as

$$\mathcal{F}_B = \left\{\varnothing \neq S \subseteq \widehat{\mathcal{R}} : \sum_{i\in S} b_i \le B\right\},$$
$$W^\star = \max_{S\in\mathcal{F}_B} C_{\text{sem}}(S). \tag{21}$$

To compute $W^\star$, the primary solver encodes every Γ_i as a C-bit mask and uses a sparse dynamic program to recover a least-cost witness. It then spends any remaining budget through density-based fill and accepts a one-for-one replacement only when the change preserves $W^\star$ and improves Φ by more than the fixed tolerance ε_{loc}.

For a matched comparison, we also apply the same fill–one-swap procedure without semantic priority. If that no-semantic result already attains $W^\star$, SILOSTITCH returns the same set unchanged. Otherwise, it refines the exact coverage witness and, when available, a coverage-optimal singleton seed, then returns the better locally refined result. Thus, when the semantic constraint is inactive, both variants return the same set; they differ only when semantic priority changes the refinement path.

D. Shared Stacker Learning

After the selector fixes $\widehat{S}$, the selected representation is

$$\phi_{\widehat{S}}(x) = \text{concat}_{i\in\widehat{S}}\, q_i(x) \in \mathbb{R}^{|\widehat{S}|(C+1)}. \tag{22}$$

Local score perturbation. When local score protection is enabled, client j independently perturbs every coordinate of each selected block:

$$\widetilde{q}_i(x_t) = q_i(x_t) + \eta_{i,t}, \qquad \eta_{i,t,k} \overset{\text{iid}}{\sim} \text{Lap}\left(0, \frac{2}{\epsilon_b}\right). \tag{23}$$

We write the perturbed representation as

$$\widetilde{\phi}_{\widehat{S}}(x) = \text{concat}_{i\in\widehat{S}}\, \widetilde{q}_i(x). \tag{24}$$

Shared stacker fitting. Using the independent stacker-fit partition $\mathcal{D}^{\text{stk}}$, the coordinator then trains

$$\widehat{f}_{\widehat{S}} = \mathsf{Fit}_{\text{global}}\left(\{(\widetilde{\phi}_{\widehat{S}}(x_t), y_t) : (x_t, y_t) \in \mathcal{D}^{\text{stk}}\}\right). \tag{25}$$

For the nonprivate configuration, the method sets $\widetilde{\phi}_{\widehat{S}} = \phi_{\widehat{S}}$. Both configurations use only the columns indexed by $\widehat{S}$. Unselected blocks therefore cannot affect the learned classifier.

The centralized sensitivity experiment samples Eq. 23 after selection to measure the effect of noise. A distributed realization samples the same noise privately at each client. At sealed-test inference, the method uses the same selected layout and leaves every contributed detector unchanged.

V. Theoretical Analysis

In this section, we establish guarantees for score-upload privacy, calibration, primary optimization, and local secondary refinement. We then analyze how client-mixture shift affects the interpretation of current evidence.

A. Local Privacy of Score Uploads

We first analyze the privacy of the score blocks randomized before upload.

We fix the selected set before observing any stacker-fit record and treat each record label as public. Accordingly, (x, y) and (x', y) are adjacent only when their feature vectors differ. This label-fixed relation is necessary because Algorithm 1 uploads y without perturbation. For a batched upload, adjacent batches have the same size, row alignment, and label sequence, and they differ in exactly one feature vector. Each client samples noise independently across records, selected detector blocks, and output coordinates.

Theorem V.1 (Label-conditional local score privacy)**.** Suppose that the selected detectors and their semantic maps are fixed and that $0 < \epsilon_b < \infty$ and each client uses fresh private randomness in Eq. 23. Then one released detector block is $(\epsilon_b, 0)$-locally differentially private under the preceding adjacency relation. If $s = |\widehat{S}|$ blocks are released for the same record, their joint release is $(s\epsilon_b, 0)$-locally differentially private. Under one-record replacement, releasing the entire client batch has the same record-level guarantee. More generally, assigning parameter ϵ_i to block i gives $(\sum_{i\in\widehat{S}} \epsilon_i, 0)$-local differential privacy.

Proof. Because $q_i(x)$ and $q_i(x')$ are probability vectors, $\|q_i(x) - q_i(x')\|_1 \le 2$. Therefore, coordinatewise Laplace noise with scale $2/\epsilon_b$ bounds the density ratio of one released block by e^{ϵ_b}. Sequential composition over the s selected blocks gives $e^{s\epsilon_b}$. Because different rows use independent randomization and neighboring batches differ in one row, parallel composition across rows leaves this bound unchanged. Appending the unchanged public labels does not alter the density ratio. □

Thus, a total per-record score budget ϵ can be obtained by assigning $\epsilon_b = \epsilon/s$ to each block, which changes the noise scale to $2s/\epsilon$. This is a label-conditional, record-level guarantee for the randomized score upload. Repeated uploads compose across rounds.

B. Calibration under Heterogeneous Clients

The privacy result concerns the fit partition, whereas detector eligibility depends on separate calibration records. We therefore analyze those statistics next.

Because calibration records can originate from different clients and classes, we condition on both the observed client–label design $\mathcal{I} = \{(j_t, y_t)\}_{t=1}^N$ and the candidate pool $\mathcal{Q}$ frozen by the preceding fit stage. We assume that calibration records are conditionally independent given $(\mathcal{Q}, \mathcal{I})$, although their conditional distributions may differ across t. Define

$$\mu_i = \frac{1}{N}\sum_{t=1}^{N} \mathbb{E}[g_{i,t} \mid \mathcal{Q}, \mathcal{I}], \qquad R_i = 1 - \mu_i, \tag{26}$$

and

$$\mu_{i,c} = \frac{1}{N_c}\sum_{t:y_t=c} \mathbb{E}[g_{i,t} \mid \mathcal{Q}, \mathcal{I}]. \tag{27}$$

Thus, these quantities describe the record-weighted calibration mixture rather than equal-client or worst-client performance.

Theorem V.2 (Simultaneous calibration)**.** Under the preceding assumptions, with probability at least $1 - \delta$, the following inequalities hold simultaneously for every candidate i and class c:

$$R_i \le R_i^+ \le R_i + 2r, \tag{28}$$

$$\max\{0, \mu_{i,c} - 2r_c\} \le a_{i,c} \le \mu_{i,c}. \tag{29}$$

Consequently, the inequalities remain valid for every candidate in any set selected adaptively from the calibration statistics.

Proof. Conditioning on $(\mathcal{Q}, \mathcal{I})$, Hoeffding's inequality gives

$$\Pr\{|\widehat{\mu}_i - \mu_i| > r \mid \mathcal{Q}, \mathcal{I}\} \le 2e^{-2Nr^2} = \frac{\delta}{2M}.$$

Therefore, a union bound over M candidates spends at most $\delta/2$. Likewise,

$$\Pr\{|\widehat{\mu}_{i,c} - \mu_{i,c}| > r_c \mid \mathcal{Q}, \mathcal{I}\} \le 2e^{-2N_c r_c^2} = \frac{\delta}{2MC},$$

so a second union bound over MC pairs spends at most $\delta/2$. The intersection has probability at least $1 - \delta$, and it does not require different candidates evaluated on the same record to be independent.

On this event, $1 - \widehat{\mu}_i + r$ lies between R_i and $R_i + 2r$. Moreover, $\widehat{\mu}_{i,c} - r_c$ lies between $\mu_{i,c} - 2r_c$ and $\mu_{i,c}$. Applying $[\cdot]_+$ proves the class inequalities. Because the event covers all $M + MC$ coordinates before selection, taking a data-dependent subset requires no additional union bound over 2^M sets. □

For any pool $\mathcal{E}$, let $W(\mathcal{E})$ be the largest feasible semantic coverage available from that pool, with value zero if no nonempty set is feasible. On the event in Theorem V.2, define

$$\mathcal{R}^- = \{i : R_i \le \tau_L - 2r\}, \qquad \mathcal{R}^0 = \{i : R_i \le \tau_L\}. \tag{30}$$

Then,

$$\mathcal{R}^- \subseteq \widehat{\mathcal{R}} \subseteq \mathcal{R}^0, \qquad W(\mathcal{R}^-) \le W(\widehat{\mathcal{R}}) \le W(\mathcal{R}^0). \tag{31}$$

Indeed, the left inclusion follows from $R_i^+ \le R_i + 2r$, while the right inclusion follows from $R_i \le R_i^+$. Thus, if one primary-optimal witness over $\mathcal{R}^0$ lies entirely in $\mathcal{R}^-$, calibration preserves the population-mixture primary value.

With $\tau_L = 1$, the upper-confidence gate acts as a finite-sample admissibility screen over normalized Brier risk. We therefore refer to $\widehat{\mathcal{R}}$ as the calibration-eligible pool.

C. Exact Primary Coverage under a Hard Budget

The simultaneous event validates the statistics used to form the eligible pool. With that pool fixed, we now characterize the primary selection problem and its exact solver.

Theorem V.3 (Primary hardness and exact parameterized solver)**.** Computing $W^\star$ is NP-hard, even when every candidate is eligible, every cost is one, and all class weights are uniform. Nevertheless, the bitmask dynamic program returns a nonempty feasible set S_0 with

$$C_{\mathrm{sem}}(S_0) = W^\star. \tag{32}$$

It stores at most 2^C masks and examines at most $|\widehat{\mathcal{R}}|2^C$ state transitions. Thus, its primary stage is fixed-parameter tractable in C.

Proof. For hardness, take a Maximum Coverage instance with universe $\{1, \dots, C\}$, subsets $\Gamma_1, \dots, \Gamma_M$, and cardinality limit k. Set $b_i = 1$, $B = k$, and $w_c = 1/C$. Maximizing C_{sem} then solves that instance up to the constant factor $1/C$.

For exactness, after processing the first t eligible candidates, associate each reachable union mask h with its least-cost witness. Omitting candidate t preserves an earlier state, whereas including it extends an earlier mask to $h \vee \Gamma_t$. Every subset makes exactly one of these choices. Moreover, a lower-cost witness for the same mask dominates every higher-cost witness under all future union operations. Induction therefore shows that the final table contains the least-cost witness for every reachable mask. Scanning the budget-feasible masks returns exactly $W^\star$.

A C-bit mask has at most 2^C values, and each eligible candidate extends each current mask at most once. When all attainable masks have value zero, the solver returns the least-cost feasible singleton, which enforces the nonempty stacker input without changing the primary value. □

The transition bound excludes witness reconstruction and sorting overhead. We restrict the evaluated solver to at most 10^6 states. The exactness guarantee applies to every instance processed within this limit.

D. Secondary Objective and Local Refinement

The primary solver determines $W^\star$. The remaining question is how the method compares feasible sets that attain this value.

Let $\mathcal{V} = \{C_{\mathrm{sem}}(S) : S \in \mathcal{F}_B\}$ contain the attainable primary values. When $\mathcal{V}$ has at least two values, let Δ_{inst} be its smallest positive gap. Because $0 \le \Phi(S) \le 1$, any coefficient $K > 1/\Delta_{\mathrm{inst}}$ is sufficient to make every maximizer of $KC_{\mathrm{sem}} + \Phi$ lexicographic for that fixed instance. Normalized class weights can make the smallest positive coverage gap arbitrarily small, so a universal finite coefficient is unavailable. The strict order in Eq. 8 does not require instance-specific scale tuning.

Proposition V.4 (Secondary structure)**.** For fixed calibration statistics, U, C_{pred}, and D_{rep} are normalized, monotone, submodular set functions. Consequently, their nonnegative weighted combination Φ has the same properties.

Proof. For U and C_{pred}, each summand applies the nondecreasing concave function h_τ to a nonnegative modular sum. Therefore, the gain from adding the same candidate cannot increase as the current set grows. For D_{rep}, target k contributes $\max_{i \in S} \kappa_{i,k}$, whose marginal gain is

$$\max\{0, \kappa_{a,k} - \max_{i \in S} \kappa_{i,k}\}.$$

This gain also cannot increase as S grows. Averaging and taking a nonnegative weighted sum preserve all three properties. □

Although Φ is monotone submodular, the equal-primary family

$$\mathcal{F}^\star = \{S \in \mathcal{F}_B : C_{\mathrm{sem}}(S) = W^\star\} \tag{33}$$

is not downward closed and need not satisfy an exchange axiom. Therefore, standard monotone-submodular knapsack guarantees, including $1 - 1/e$, do not apply to the secondary refinement stage.

For $S \in \mathcal{F}^\star$, let $\mathcal{N}_1(S)$ contain every set obtained by removing exactly one selected candidate and adding exactly one unselected candidate while preserving budget feasibility and $W^\star$.

Theorem V.5 (Coverage preservation and one-swap certificate)**.** Given an exact primary solution, the refinement procedure terminates after finitely many accepted operations and returns $\widehat{S} \in \mathcal{F}^\star$ such that

$$\Phi(S') \le \Phi(\widehat{S}) + \varepsilon_{\mathrm{loc}}, \qquad \forall S' \in \mathcal{N}_1(\widehat{S}). \tag{34}$$

Proof. The constrained seed has coverage $W^\star$. Because C_{sem} is monotone and $W^\star$ is already maximal, every feasible fill keeps that value. The algorithm checks every accepted replacement for budget feasibility and equality to $W^\star$. Each accepted operation also increases Φ by more than $\varepsilon_{\mathrm{loc}}$. Because the candidate pool has only finitely many subsets, the procedure terminates. At termination, the exhaustive one-for-one search has found no feasible coverage-preserving replacement whose gain exceeds the tolerance, which proves Eq. 34. □

The certificate covers the one-swap neighborhood explored by the refinement procedure. The fill stage ranks candidates by gain per byte and stops when the chosen candidate's absolute gain falls below tolerance; broader additions and multi-candidate exchanges remain separate optimization choices.

E. Effective Coverage under Client Mixture Shift

The preceding results treat declared support and current predictive behavior as separate quantities. We now ask when the former is backed by the latter and how that relation changes under client-mixture shift.

We use the following quantities as post-selection certificates for the frozen returned set, reusing statistics already computed during calibration.

To test whether declared source support is backed by current evidence, let $\vartheta_c \in [0,1]$ be a prespecified current-competence threshold and define

$$C_{\mathrm{cert}}(S;\boldsymbol{\vartheta}) = \sum_{c=1}^{C} w_c \mathbf{1}\{\exists i \in S : \Gamma_{i,c} = 1 \wedge a_{i,c} \geq \vartheta_c\}. \quad (35)$$

Thus, a class contributes only when a selected detector provides both declared source support and a conservative current-utility score. For comparison, let $C^a_\mu(S;\boldsymbol{\vartheta})$ replace $a_{i,c} \geq \vartheta_c$ by $\mu^a_{i,c} \geq \vartheta_c$, where $a \in \{\mathrm{cal}, \mathrm{dep}\}$.

Let $h = (j,c)$ index a client–class cell. Suppose that detector behavior within each cell remains stable between calibration and deployment, while only the mixture weights change. Let $R_{i,h} \in [0,1]$ denote the cell risk, and let π^{cal}_h and π^{dep}_h denote the two mixtures. For $a \in \{\mathrm{cal}, \mathrm{dep}\}$, define

$$R^a_i = \sum_{j,c} \pi^a_{j,c} R_{i,j,c}, \qquad \mu^a_{i,c} = \sum_j \pi^a_{j|c}(1 - R_{i,j,c}), \quad (36)$$

and let $\mathrm{TV}(p,q) = \frac{1}{2}\|p - q\|_1$. Under the calibration design, the means in Eqs. 26–27 equal $1 - R^{\mathrm{cal}}_i$ and $\mu^{\mathrm{cal}}_{i,c}$, respectively.

Theorem V.6 (Effective coverage under mixture shift)**.** Suppose that the within-class client mixtures differ by at most η_c. On the simultaneous calibration event, every adaptively selected set $\widehat{S}$ satisfies

$$\begin{aligned} C^{\mathrm{cal}}_\mu(\widehat{S};\boldsymbol{\vartheta} + \boldsymbol{\eta} + 2\boldsymbol{r}) &\leq C_{\mathrm{cert}}(\widehat{S};\boldsymbol{\vartheta} + \boldsymbol{\eta}) \\ &\leq C^{\mathrm{dep}}_\mu(\widehat{S};\boldsymbol{\vartheta}) \leq C_{\mathrm{sem}}(\widehat{S}). \end{aligned} \quad (37)$$

where $(\boldsymbol{r})_c = r_c$ and $(\boldsymbol{\eta})_c = \eta_c$.

Moreover, if $\mathrm{TV}(\pi^{\mathrm{dep}}, \pi^{\mathrm{cal}}) \leq \eta$, then

$$R^{\mathrm{dep}}_i \leq R^{\mathrm{cal}}_i + \eta. \quad (38)$$

Proof. Theorem V.2 and the mixture assumption give

$$\mu^{\mathrm{cal}}_{i,c} \geq \vartheta_c + \eta_c + 2r_c \Longrightarrow a_{i,c} \geq \vartheta_c + \eta_c \Longrightarrow \mu^{\mathrm{dep}}_{i,c} \geq \vartheta_c.$$

Intersecting these implications with the fixed support edge $\Gamma_{i,c} = 1$, taking the existential quantifier over selected candidates, and summing class weights proves Eq. 37.

For the final claim, total variation bounds the change in the expectation of any $[0,1]$-valued function. Applying this bound to the cell risks yields Eq. 38. □

Thus, C_{cert} connects the historical class support of selected detectors with current class-conditional Brier utility under client-mixture shift. The executed selector uses C_{sem} and Φ; C_{cert} and the mixture-sensitivity bound are derived analysis quantities for the frozen returned set.

VI. Evaluation

We evaluate whether SiloStitch preserves source class support under a hard selection cost budget while retaining predictive quality. We organize the evaluation around five questions:

- **RQ1: Detection quality under a budget.** How much predictive quality does the shared detector retain as the selection budget decreases?
- **RQ2: Client label heterogeneity.** How do different client label distributions affect usable training partitions, class support, and detection quality?
- **RQ3: Selection signals and learners.** Which selection signals, semantic settings, and learning backends change the result?
- **RQ4: Score randomization.** How much detection quality remains as Laplace perturbation becomes stronger?
- **RQ5: Scalability and runtime.** How do selection time and communication grow with the deployment?

A. Experimental Setup

Datasets. We use four multiclass cybersecurity datasets: CIC DDoS2019 (DDoS) [23], CICMalDroid2020 (MalDroid) [24], CIC Darknet2020 (Darknet) [25], and CIRA CIC DoHBrw 2020 (DoH) [26]. We use 66 clients in every task and let each client contribute one candidate detector. Every client still receives from 74 to 30,517 records for local detector fitting and at least two classes, so all 66 candidates remain trainable. We keep this common federation size instead of tuning it separately for each dataset. As a result, differences among datasets cannot arise merely from a larger or smaller detector pool, and the central 50% operating point asks the coordinator to choose roughly 33 detectors from 66. In RQ5, we separately vary the candidate pool from 16 to 256 to test how selection scales. Table II summarizes the evaluation scale. The reserved DDoS test set contains 13 observed classes, including one attack class absent from client training. That class contributes to macro F1 but not to minimum evaluated class recall.

Protocol. We divide each training pool into disjoint 60%, 10%, and 30% partitions for local detector training, selection validation, and shared stacker training. We reserve the test set until both selection and stacker fitting have finished. Unless stated otherwise, we use five repeated splits, LightGBM [27] for local detectors and the shared stacker, a 50% selection cost budget, minimum source support $n_{\min} = 2$, class weights that are inversely proportional to the square root of class frequency, and equal weights for the three secondary selection signals.

Baselines and reference point. We compare SiloStitch with two baselines under the same selection cost budget. For the random baseline, each of 20 fixed selection seeds within a data split generates a random candidate order. The coordinator scans that order once and adds a detector whenever it fits the remaining budget. We average the 20 selections within the split. For the individual utility baseline, the coordinator ranks detectors by mean calibrated Brier utility per selection cost byte and greedily packs the fixed order. We use the full pool of 66 detectors as the unconstrained reference and the 100% selection cost reference point. By contrast, SiloStitch first maximizes weighted source class support and then refines current predictive quality without reducing that support. We later remove only the source support priority while retaining the same secondary search.

Metrics and statistics. We choose macro F1 as the main detection score because it gives every class equal weight. In imbalanced security traffic, this prevents a frequent class

Table II
WE EVALUATE FOUR DATASETS WITH DIFFERENT SAMPLE, FEATURE, AND CLASS COUNTS.

Dataset	Training samples	Test samples	Features	Evaluated classes
DDoS	3,355,966	1,127,526	79	13
MalDroid	8,118	3,480	470	5
Darknet	99,066	42,459	76	11
DoH	1,005,708	431,034	29	4

from hiding weak detection of a rarer attack class. We also report accuracy for overall correctness, balanced accuracy for average class recall, and minimum evaluated class recall. The last metric is the lowest recall among labels that appear in the sealed test set and belong to the fixed training class set. We measure weighted class coverage separately because it describes retained training support rather than classifier quality. For the DDoS privacy experiment, we report *attack leakage*, which is the fraction of predictions labeled benign that are actually attacks. Equation 5 defines the selection budget. For system results, we add the initial candidate upload and stacker label upload according to Eq. 7.

We report means and 95% Student t confidence intervals over five repeated splits unless stated otherwise. When two conditions share a split, we pair their observations. We apply Holm correction within each comparison family fixed before analysis. We recompute every detection score from the reserved test confusion matrix.

Experimental environment. We run a centralized simulation on one machine under Linux and Python 3.9.25. The software stack uses NumPy 1.26.4, Scikit Learn 1.3.2, LightGBM 4.6.0, and XGBoost [28] 1.4.2. We record wall clock time and the logical bytes exchanged by the evaluated protocol.

B. RQ1: Detection Quality under a Budget

To determine whether SILOSTITCH retains detection signal under a hard budget, we first compare macro F1 at the central 50% operating point. We then inspect accuracy and minimum evaluated class recall at the same point. Finally, we trace macro F1 across the full budget range.

At the 50% target, SILOSTITCH uses from 49.5% to 50.0% of the full pool selection cost. It keeps macro F1 within 0.0052 of the full pool on MalDroid, Darknet, and DoH. DDoS shows the larger gap of 0.0258. As Fig. 2 shows, adding models does not always produce a monotonic gain. MalDroid improves as the budget grows, whereas the other datasets fluctuate within narrower ranges.

Across MalDroid, Darknet, and DoH, selecting half of the detectors changes accuracy by at most 0.0046 and minimum evaluated class recall by at most 0.0061. On DDoS, SILOSTITCH loses 0.0296 accuracy but only 0.0094 minimum evaluated class recall. On Darknet, even the full pool reaches only 0.0077 minimum evaluated class recall, compared with 0.0058 after selection. Both configurations therefore reveal the same Darknet bottleneck. Across all four datasets, SILOSTITCH also preserves the full attainable weighted class support.

Against random selection, SILOSTITCH raises mean macro F1 on DDoS, MalDroid, and Darknet, while the two means differ by 0.0010 on DoH. Compared with individual utility ranking, SILOSTITCH raises the mean on MalDroid and Darknet and differs by 0.0005 on DoH. On DDoS, individual utility ranking gains 0.0150 macro F1 at the 50% budget but retains only 0.910 of the attainable weighted source support. At the same budget, SILOSTITCH attains the full support value of 1.0. Thus, the DDoS result exposes the intended tradeoff between average predictive quality and weighted source support.

C. RQ2: Client Label Heterogeneity

When clients observe different attack classes, we evaluate both naturally occurring and controlled label distributions. To preserve the original deployment structure, we first keep the original DDoS and Darknet client assignments and repeat only the data split. To isolate label imbalance, we then draw client partitions with Dirichlet parameters $\alpha \in \{10, 1, 0.5, 0.1\}$ and compare them with an independent and identically distributed allocation. We require every accepted partition to provide at least two local training classes to each client so that each local detector remains a multiclass model.

Figure 3 summarizes both partition feasibility and detection quality as client label imbalance increases.

On Darknet, all ten partitions remain usable at every level, while measured divergence rises from 0.0008 under the balanced allocation to 0.3516 at $\alpha = 0.1$. On MalDroid, all ten balanced and all ten $\alpha = 10$ partitions remain usable, but only five $\alpha = 1$ partitions pass the same rule. None of the ten fixed partition seeds at $\alpha = 0.5$ or 0.1 can divide this smaller five class training pool among 66 clients while retaining two classes per client, despite allowing up to 100 generation attempts per seed.

Within the usable range, Darknet macro F1 stays between 0.5895 and 0.5976 at the 50% budget as imbalance grows. MalDroid stays between 0.8328 and 0.8375. In a separate budget sweep that retains the original client assignments, a larger budget helps the weakest class more clearly. As the budget rises from 25% to 75%, Darknet macro F1 grows from 0.6162 to 0.6290 and minimum evaluated class recall grows from 0.0452 to 0.0964. Over the same range, DDoS macro F1 changes from 0.6039 to 0.6104 and minimum evaluated class recall changes from 0.0568 to 0.0773.

We then remove the source support priority while retaining the same secondary search. Across 30 comparisons with the original assignments and 225 comparisons with controlled assignments, both versions choose the same set and obtain

Table III
WE COMPARE MACRO F1 AGAINST TWO BUDGET MATCHED BASELINES AT THE 50% COMMUNICATION BUDGET. THE FULL DETECTOR POOL PROVIDES THE UNCONSTRAINED REFERENCE. VALUES REPORT MEANS AND 95% CONFIDENCE INTERVALS OVER FIVE REPEATED SPLITS.

Method	DDoS	MalDroid	Darknet	DoH
Full detector pool	0.6412 ± 0.0121	0.8449 ± 0.0123	0.5972 ± 0.0128	0.6073 ± 0.0042
Random selection	0.6079 ± 0.0287	0.8344 ± 0.0100	0.5931 ± 0.0071	0.6074 ± 0.0028
Individual utility ranking	0.6303 ± 0.0437	0.8374 ± 0.0132	0.5921 ± 0.0168	0.6069 ± 0.0042
SILOSTITCH	0.6153 ± 0.0453	0.8397 ± 0.0121	0.5953 ± 0.0148	0.6064 ± 0.0038

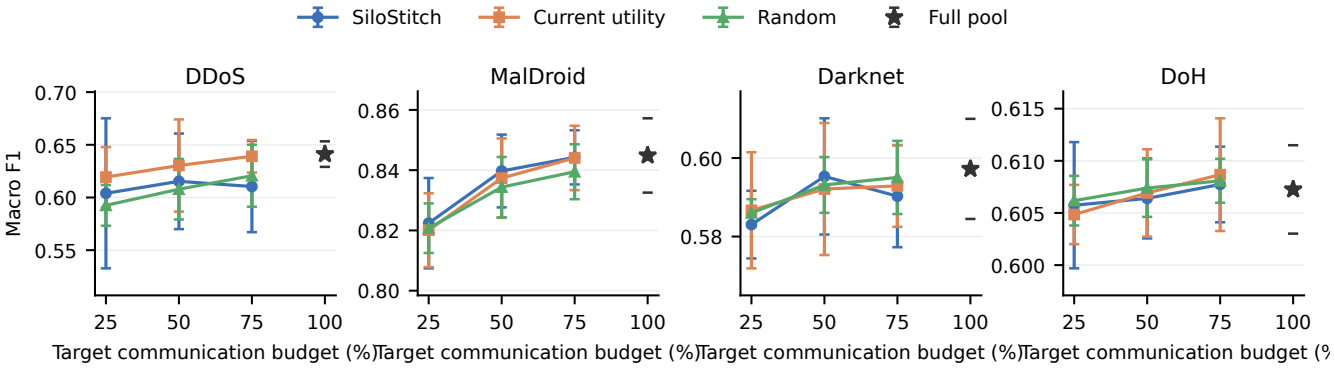

Figure 2. The figure tracks detection quality as the selection cost budget changes. Lines show three selection rules at the 25%, 50%, and 75% budgets. The star marks the full pool of 66 models. Error bars show 95% confidence intervals over five repeated splits. Each dataset uses its own vertical scale.

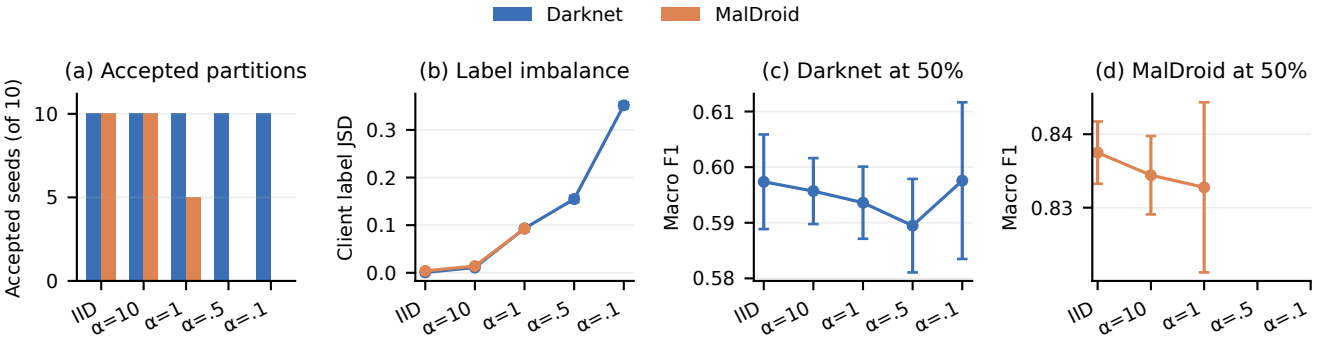

Figure 3. The figure shows how controlled client label imbalance changes feasibility and detection. Panel (a) counts valid partitions among ten fixed seeds. Panel (b) measures label imbalance with Jensen Shannon divergence. Panels (c) and (d) report macro F1 at the 50% budget. Error bars span valid partition seeds, with $n = 10$ except for MalDroid at $\alpha = 1$, where $n = 5$. Missing MalDroid points denote partition settings that cannot train every client on at least two classes.

the same macro F1, minimum evaluated class recall, communication, and attained support. In these evaluated layouts, the secondary refinement already reaches the maximum attainable support.

D. RQ3: Selection Signals and Learners

After the primary objective fixes maximum class support, SILOSTITCH uses current evidence to order the remaining feasible sets. We remove individual detector quality, classwise quality, and prediction diversity one at a time. Because every variant still attains the maximum support, this comparison isolates how the three signals order otherwise feasible detector sets.

At the 50% budget, removing individual detector quality lowers DDoS macro F1 by 0.0190. Removing prediction diversity instead raises it by 0.0181 on the same dataset. The direction changes across datasets and budgets. After Holm correction, no removal produces a statistically significant change. The three signals therefore work as a joint ranking rule rather than as independent sources of universal improvement.

We also vary $n_{\min} \in \{1, 2, 5, 10\}$ and compare class weights based on the inverse square root of frequency with uniform weights. Across two fixed Darknet layouts and two learner combinations, these choices leave the selected set, support, macro F1, and minimum evaluated class recall unchanged. By contrast, the learning backends produce visible

Table IV
THE TABLE COMPARES MACRO F1 ACROSS LOCAL AND SHARED LEARNERS. VALUES ARE MEANS OVER FIVE SPLITS. BOLD MARKS THE LARGEST MEAN FOR EACH DATASET.

Dataset	XGB →XGB	LGBM →XGB	XGB →LGBM	LGBM →LGBM
DDoS	0.617	**0.635**	0.633	0.615
MalDroid	0.806	0.833	0.827	**0.840**
Darknet	0.578	0.585	0.592	**0.595**
DoH	0.592	0.591	**0.607**	0.606

score differences, as Table IV shows.

On DDoS, LightGBM local detectors with an XGBoost shared learner obtain the largest mean. On MalDroid and Darknet, LightGBM obtains the largest mean at both levels. On DoH, XGBoost local detectors with a LightGBM shared learner lead by a small margin. After Holm correction, only the two DoH configurations with an XGBoost shared learner remain below the LightGBM alternatives. The learning backend is therefore the clearest practical tuning choice in this experiment.

E. RQ4: Score Randomization

To measure the utility cost of score randomization, we freeze the selected set and add independent Laplace noise with scale $2/\epsilon$ to every score block used for shared stacker training. A smaller ϵ adds stronger noise. Here ϵ is the privacy parameter for one released score block. Under the public label neighboring relation in Theorem V.1, two neighboring records share a label and differ in one feature vector. The theorem gives label conditional record level local differential privacy to each released block. When one record contributes s released blocks, sequential composition gives a total parameter of $s\epsilon$ for that upload. During the centralized sensitivity experiment, we sample the same noise distribution after selection and keep the test encoding fixed so that the curves isolate the effect on stacker training.

Without added noise, the selected half and the full pool obtain macro F1 scores of 0.6153 and 0.6412. At the strongest evaluated setting, $\epsilon = 1$, they obtain 0.5105 and 0.5298. The selected half loses 0.1048, while the full pool loses 0.1114. Both configurations therefore retain a similar fraction of their original detection score as perturbation grows.

When we inspect attack leakage, the curve reveals a sharper operational change. For the selected half, it rises from 0.055 without noise to 0.262 at $\epsilon = 5$ and 0.583 at $\epsilon = 1$. For the full pool, it rises from 0.056 to 0.193 and then 0.644. At the strongest perturbation, more than half of the records predicted as benign are attacks. By reporting both macro F1 and attack leakage, the evaluation shows how a moderate aggregate score can hide a costly security failure.

F. RQ5: Scalability and Runtime

To verify exact selection directly, we compare SILOSTITCH with exhaustive search at $C = 8$ and $M \in \{8, 10, 12, 16\}$. Across ten repetitions at every size, the solver matches the exhaustive optimum in all 40 comparisons. Its median time ranges from 37.70 to 118.72 ms, while every 95th percentile remains below 125 ms. We then vary the number of candidates and attack classes separately in Fig. 5.

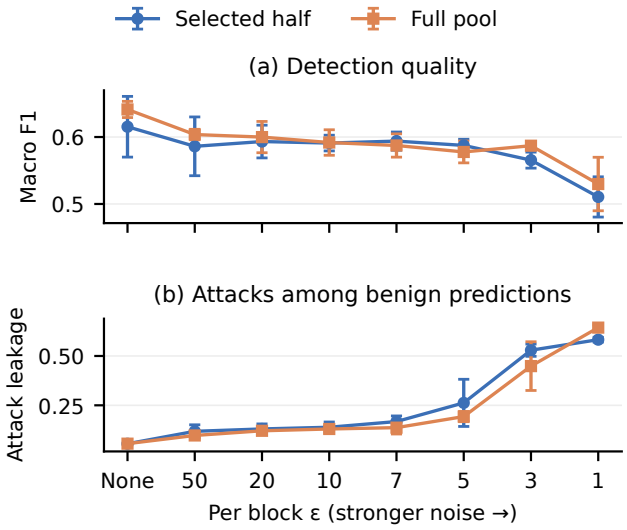

Figure 4. The figure shows how Laplace score randomization changes detection. Panel (a) reports macro F1. Panel (b) reports attacks hidden among predictions labeled benign. Larger ϵ adds less noise. Error bars show 95% confidence intervals over the same five DDoS splits.

With 12 attack classes, SILOSTITCH selects from 66 candidates in a median of 1.99 seconds and from 256 candidates in 33.73 seconds. With 66 candidates, raising the attack vocabulary from 12 to 20 classes increases the median from 2.95 to 100.10 seconds. Within this range, attack class diversity drives time more strongly than the number of candidate models.

To connect runtime and communication savings with detection quality, we report the combined result in Table V. The runtime column includes every evaluated rule and budget within one data split.

Across the four tasks, SILOSTITCH reduces total logical communication by between 49.3% and 50.4% while losing at most 0.0258 macro F1. Across MalDroid, Darknet, and DoH, the loss remains at or below 0.0052. For DDoS, the larger quality change still preserves the full attainable weighted class support reported in RQ1. Therefore, the 50% selection cost operating point yields an approximately one half reduction in total logical communication, with a small quality change on three tasks and a measurable tradeoff on DDoS.

When one split evaluates every rule and budget, its mean duration ranges from 0.78 to 50.09 minutes. For MalDroid and Darknet, stacker training dominates the total runtime. By contrast, selection takes the largest share on DDoS. Therefore, the main computation cost depends on the task rather than on selection alone. At the scale of 66 candidates, the exact selector itself finishes in seconds.

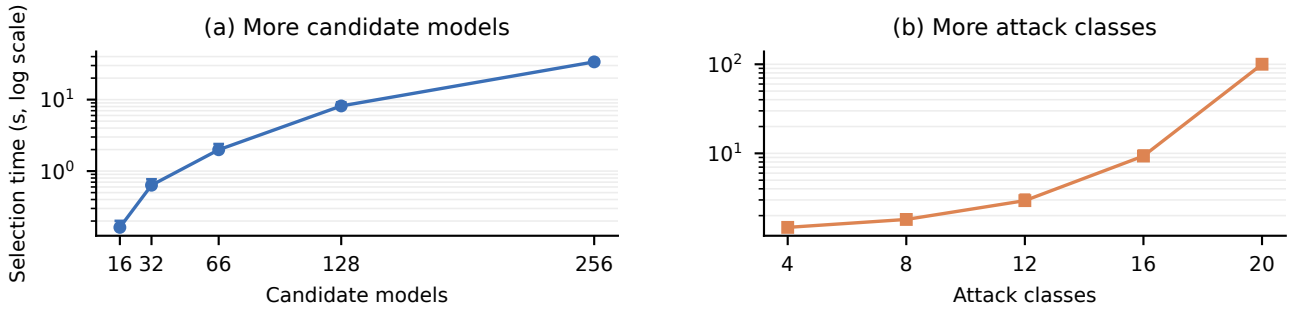

Figure 5. The figure shows how selection time grows with the deployment. The left panel varies candidate models, while the right panel varies attack classes. Each panel uses an independently sampled sweep. Points show medians over ten repetitions, and upper bars mark the 95th percentile.

Table V
WE REPORT THE TIME REQUIRED TO EVALUATE EVERY SELECTION RULE AND BUDGET ON ONE MACHINE, TOGETHER WITH TOTAL LOGICAL COMMUNICATION AT THE 50% SELECTION COST BUDGET. TIMES ARE MEANS OVER FIVE SPLITS. THE FINAL COLUMN REPORTS THE MACRO F1 LOSS RELATIVE TO THE FULL DETECTOR POOL.

Dataset	Runtime (min)	Dominant stage	Full pool (MiB)	SILOSTITCH (MiB)	Communication saved	Macro F1 loss
DDoS	50.09	Selection (34.2%)	7,040.05	3,489.07	50.4%	0.0258
MalDroid	0.78	Stacker training (74.9%)	662.66	335.94	49.3%	0.0052
Darknet	42.51	Stacker training (89.1%)	3,436.78	1,732.29	49.6%	0.0019
DoH	8.54	Stacker training (33.2%)	2,942.76	1,488.51	49.4%	0.0009

VII. DISCUSSION

The lexicographic objective answers two distinct deployment questions. Weighted semantic coverage specifies which source-side class support should remain whenever the budget permits. The secondary score distinguishes current prediction behavior only after the best attainable coverage has been fixed. An instance-specific scalarization can reproduce this order when its minimum positive coverage gap is known, but no universal finite coefficient works across arbitrary class weights and candidate pools. The ordered formulation therefore makes the intended priority explicit. Semantic coverage measures preserved source-side breadth, while class-conditional Brier lower bounds and sealed-test metrics measure current competence. The effective-coverage certificate connects these two views by requiring a selected, source-supported detector to exceed a deployment-specific class-utility threshold.

We set $\tau_L = 1$, so the upper-confidence gate acts as a finite-sample admissibility screen over normalized Brier risk. The simultaneous calibration event remains valid after adaptive selection. Within the resulting pool, the bitmask dynamic program computes the exact primary value. Subsequent additions and single replacements preserve that value while providing a one-swap local certificate.

Client heterogeneity determines the population described by the calibration statistics. Our experiments weight the mixture over client and class cells by their record counts. Under stable cell-conditional risks and a deployment mixture within total variation η of the calibration mixture, deployment Brier risk is at most calibration risk plus η. The logical-byte budget then measures model delivery and aligned score communication consistently across all configurations.

The privacy protocol asks each client to perturb selected score blocks with fresh private Laplace randomness before upload. The theorem assigns ϵ_b to one block and composes to $s\epsilon_b$ for s selected blocks of one record. Our centralized sensitivity experiment samples the same post-selection distribution and measures its utility over seven perturbation levels. A distributed realization places this randomization at each client as specified by the protocol.

VIII. CONCLUSION

In summary, SILOSTITCH integrates pretrained intrusion detectors under a hard logical-byte budget without changing their parameters. It aligns their outputs, screens candidates with held-out calibration data, uses exact dynamic programming to maximize weighted semantic coverage, and refines current utility and representativeness while preserving that value. Across four datasets, the 50% target budget cuts logical payload by 49.3–50.4% and closely tracks all-encoder Macro-F1 on MALDROID, DARKNET, and DOH. The selected detectors feed one shared stacker, and client-side Laplace randomization gives label-conditional local privacy to uploaded score blocks. Together, the calibration, solver, privacy, and client-mixture results make post-training detector integration a principled choice under deployment constraints.

REFERENCES

[1] Australian Signals Directorate's Australian Cyber Security Centre, "Best practices for event logging and threat detection," Cyber.gov.au, 2024. [Online]. Available: https://www.cyber.gov.au/business-government/

detecting-responding-to-threats/event-logging/best-practices-for-event-logging-and-threat-detection

[2] Microsoft, "Microsoft Entra data retention," Microsoft Learn, 2026, accessed: Aug. 10, 2026. [Online]. Available: https://learn.microsoft.com/en-us/entra/identity/monitoring-health/reference-reports-data-retention

[3] L. Yang, W. Guo, Q. Hao, A. Ciptadi, A. Ahmadzadeh, X. Xing, and G. Wang, "CADE: Detecting and explaining concept drift samples for security applications," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2021, pp. 2327–2344. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/yang-limin

[4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[5] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze, Eds., vol. 2. MLSys, 2020, pp. 429–450. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html

[6] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 6357–6368. [Online]. Available: https://proceedings.mlr.press/v139/li21h.html

[7] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[8] V. Pourahmadi, H. A. Alameddine, M. A. Salahuddin, and R. Boutaba, "Spotting anomalies at the edge: Outlier exposure-based cross-silo federated learning for DDoS detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 4002–4015, 2023.

[9] J. Xu, C. Li, Y. Zheng, and Z. Li, "ENTENTE: Cross-silo intrusion detection on network log graphs with federated learning," in *Network and Distributed System Security Symposium*, 2026. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2026-s93-paper.pdf

[10] Y. Qing, Q. Yin, X. Deng, Y. Chen, Z. Liu, K. Sun, K. Xu, J. Zhang, and Q. Li, "Low-quality training data only? a robust framework for detecting encrypted malicious network traffic," in *Network and Distributed System Security Symposium*, 2024. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2024-81-paper.pdf

[11] R. Xie, J. Cao, E. Dong, M. Xu, K. Sun, Q. Li, L. Shen, and M. Zhang, "Rosetta: Enabling robust TLS encrypted traffic classification in diverse network environments with TCP-Aware traffic augmentation," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, 2023, pp. 625–642. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/xie

[12] S. Yang, X. Zheng, J. Li, J. Xu, and E. C. H. Ngai, "CoLD: Collaborative label denoising framework for network intrusion detection," in *Network and Distributed System Security Symposium*, 2026. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2026-s1950-paper.pdf

[13] Q. Dong, A. Muhammad, F. Zhou, C. Xie, T. Hu, Y. Yang, S.-H. Bae, and Z. Li, "ZooD: Exploiting model zoo for out-of-distribution generalization," in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/cd305fdee96836d5cc1de94577d71b61-Abstract-Conference.html

[14] Y. Wei, Z. Hu, L. Shen, Z. Wang, Y. Li, C. Yuan, and D. Tao, "Task groupings regularization: Data-free meta-learning with heterogeneous pre-trained models," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 52 573–52 587. [Online]. Available: https://proceedings.mlr.press/v235/wei24e.html

[15] X. Zhao, G. Sun, R. Cai, Y. Zhou, P. Li, P. Wang, B. Tan, Y. He, L. Chen, Y. Liang, B. Chen, B. Yuan, H. Wang, A. Li, Z. Wang, and T. Chen, "Model-GLUE: Democratized LLM scaling for a large model zoo in the wild," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://openreview.net/forum?id=M5JW7O9vc7

[16] S. Thirumuruganathan, M. Nabeel, E. Choo, I. Khalil, and T. Yu, "SIRAJ: A unified framework for aggregation of malicious entity detectors," in *43rd IEEE Symposium on Security and Privacy*. IEEE, 2022, pp. 507–521.

[17] W. Qiu, Y. Feng, Y. Li, Y. Chang, K. Qian, B. Hu, Y. Yamamoto, and B. W. Schuller, "Fed-MStacking: Heterogeneous federated learning with stacking misaligned labels for abnormal heart sound detection," *IEEE Journal of Biomedical and Health Informatics*, vol. 28, no. 9, pp. 5055–5066, Sep. 2024.

[18] Y. Birman, S. Hindi, G. Katz, and A. Shabtai, "Cost-effective ensemble models selection using deep reinforcement learning," *Information Fusion*, vol. 77, pp. 133–148, 2022.

[19] T. Qi, F. Wu, C. Wu, L. He, Y. Huang, and X. Xie, "Differentially private knowledge transfer for federated learning," *Nature Communications*, vol. 14, p. 3785, 2023. [Online]. Available: https://www.nature.com/articles/s41467-023-38794-x

[20] S. Maddock, G. Cormode, T. Wang, C. Maple, and S. Jha, "Federated boosted decision trees with differential privacy," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 2249–2263. [Online]. Available: https://doi.org/10.1145/3548606.3560687

[21] J. C. Duchi, M. I. Jordan, and M. J. Wainwright, "Local privacy and statistical minimax rates," in *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*, 2013, pp. 429–438.

[22] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of cryptography conference*. Springer, 2006, pp. 265–284.

[23] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (ddos) attack dataset and taxonomy," in *2019 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 2019, pp. 1–8.

[24] S. Mahdavifar, A. F. A. Kadir, R. Fatemi, D. Alhadidi, and A. A. Ghorbani, "Dynamic android malware category classification using semi-supervised deep learning," in *IEEE International Conference on Dependable, Autonomic and Secure Computing*. IEEE, 2020, pp. 515–522.

[25] A. Habibi Lashkari, G. Kaur, and A. Rahali, "Didarknet: A contemporary approach to detect and characterize the darknet traffic using deep image learning," in *2020 the 10th International Conference on Communication and Network Security*, 2020, pp. 1–13.

[26] M. MontazeriShatoori, L. Davidson, G. Kaur, and A. H. Lashkari, "Detection of doh tunnels using time-series classification of encrypted traffic," in *2020 IEEE International Conference on Dependable, Autonomic and Secure Computing*. IEEE, 2020, pp. 63–70.

[27] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," *Advances in neural information processing systems*, vol. 30, pp. 3146–3154, 2017.

[28] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.